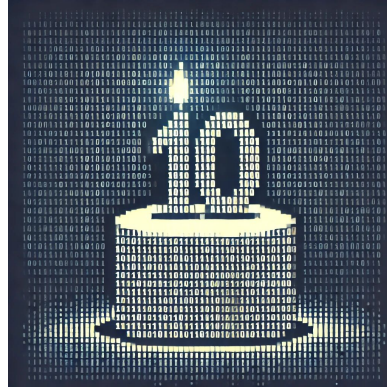


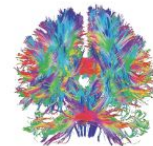
# Réunion Interflop



Verificarlo fête ses 10 ans !

Rétrospective des changements majeurs durant le projet Interflop

Yohan Chatelain  
Chercheur postdoctoral  
Centre for Addictions and Mental Health



**camh**  
Krembil Centre for  
Neuroinformatics

# Verificarlo

- Outil pour l'étude des comportements numériques des codes de calculs
- Projet open-source (Apache-2.0 with LLVM-exception)
- 15 contributeurs sur GitHub
- Première release il y a 10 ans !
  - v0.0.1 [2015] -> **Architecture de base de Verificarlo**
  - v0.2.3 [2019] -> **Première API Interflop et nouveaux backends**
  - v1.0.0 [2023] -> **Support Aarch64 ARM**
  - v2.0.0 [2024] -> **Nouvelle API Interflop**
  - v2.1.0 [2025] -> **PRISM : backend RR entièrement vectorisé**



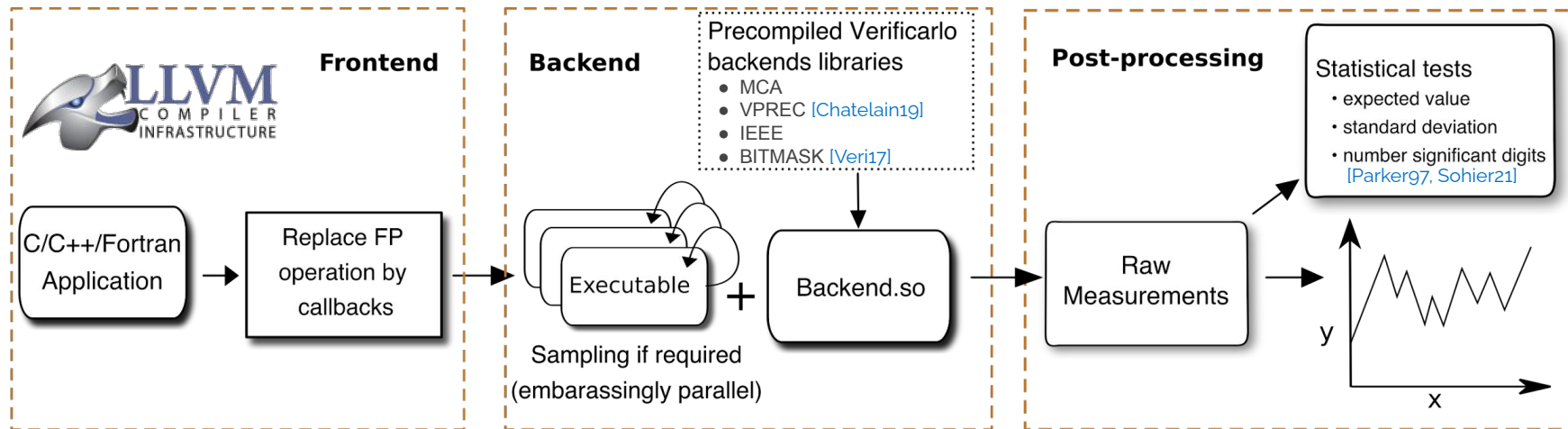
[github.com/verificarlo/verificarlo](https://github.com/verificarlo/verificarlo)

**v0.0.1**

# **Architecture de Verificarlo**



# ✓verificarlo [Veri15, Veri17, Chatelain19, Sohier21]



- Compilateur basé sur clang (C,C++) / flang (Fortran)
- Remplace les opérations flottantes par des appels de fonction génériques
- Sélection des modèles flottants alternatifs (**backend**) à l'exécution
- Linux / x86-64 architecture

**v1.0.0**

**Support Aarch64**



# Support des architectures ARM

**Précision Quadruple :** `__float128` non supporté sur ARM.

Solution : Remplacement par `_Float128` et utilisation du *shim* PortableFP128.

**Fonctions Variadiques :** Conventions d'appel différentes sur ARM64 provoquant des segfaults.

Solution : Correction de l'usage de `va_start` et alignement des signatures LLVM IR.

**Vecteurs LLVM & Toolchain :** Extensions vectorielles et incompatibilités d'outils (libomp, llvm-addr2line).

Solution : Adaptation du frontend Verificarlo et configuration adaptées à l'architecture (`#ifdef`)

Validation initiale sur Ampere Altra Q80-30 (CALMIP)



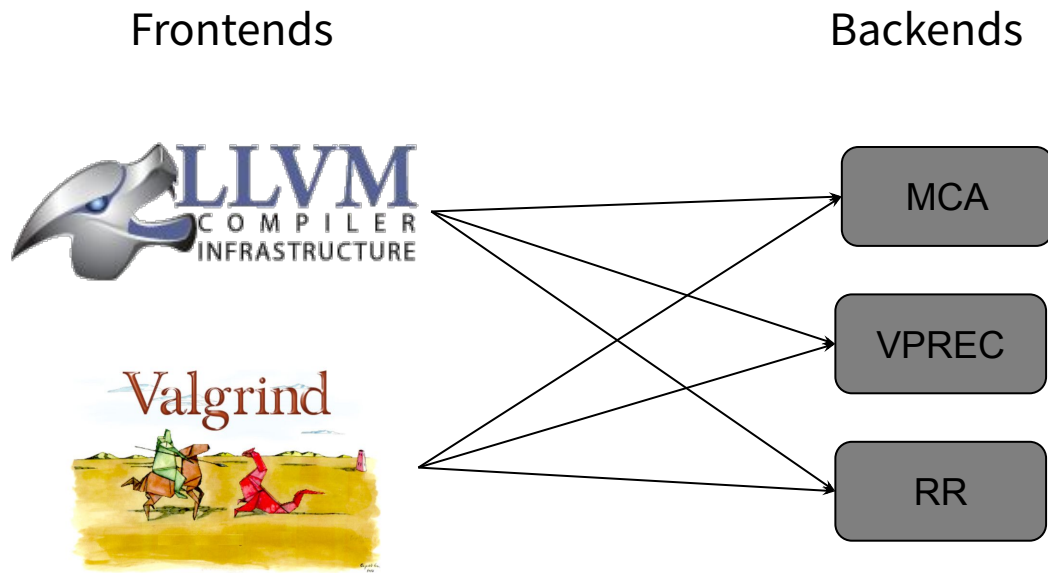
**v2.0.0**

**Nouvel API Interflop**



# Nouvelle API Interflop

Objectif : Interface commune entre les différents **frontends** et **backends**



✓erificarlo





# Verrou [Fevotte15]

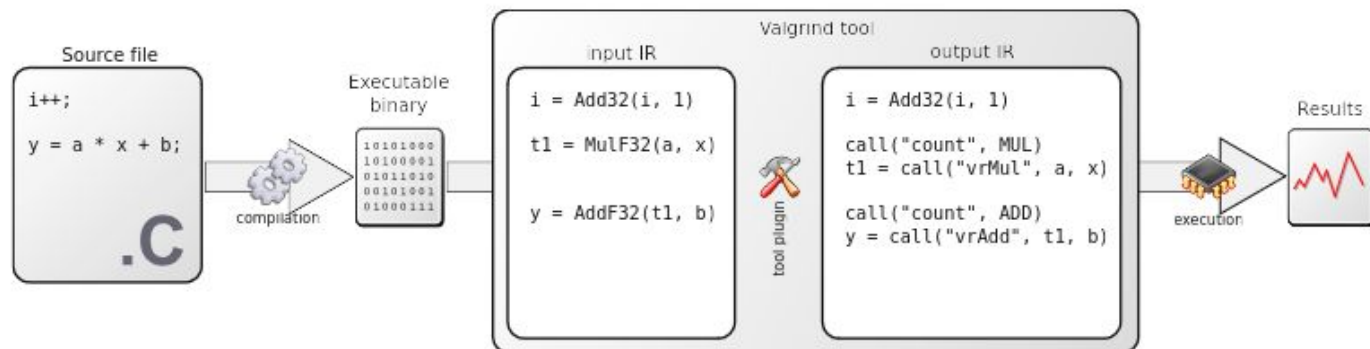


Figure 1: Schematic description of dynamic binary analysis with Valgrind

- Outil Valgrind qui prend en entrée des exécutables
- Instrumente les opérations flottantes à l'exécution (Machine Virtuelle Valgrind)
- Plusieurs backends d'arrondie stochastique disponibles

# Problèmes

1. glibc non utilisable dans Valgrind (malloc, free, ...)
2. Factorisation du code en commun (manipulation flottant, FMA, logger, RNG)
3. Pas de variables thread local dans Valgrind
4. Passage des arguments aux backends (pas de argp dans Valgrind)

# Problèmes

1. glibc non utilisable dans Valgrind (malloc, free, ...)
2. Factorisation du code en commun (manipulation flottant, FMA, logger, RNG)
3. Pas de variables thread local dans Valgrind
4. Passage des arguments aux backends (pas de argp dans Valgrind)

# Interfaçage de la glibc : interflop-stdlib

Interface commune des fonctions de la glibc utilisées dans Verificarlo et Verrou

1. Type abstrait pour chacune des fonctions (uniformisation signatures)
2. Une variable globale (pointeur de fonction) initialisée à NULL
3. Un handler pour initialiser les fonctions depuis les **frontends**

# Interfaçage de la glibc : interflop-stdlib

Interface commune des fonctions de la glibc utilisées dans Verificarlo et Verrou

1. **Type abstrait pour chacune des fonctions (uniformisation signatures)**
2. Une variable globale (pointeur de fonction) initialisée à NULL
3. Un handler pour initialiser les fonctions depuis les **frontends**

## 1. Type abstrait pour chacune des fonctions (uniformisation signatures)

```
#define EXIT_FAILURE 1

#define INTERFLOP_CHECK_IMPL(name) \
    if (interflop_##name == Null) { \
        interflop_panic("Interflop backend error: " #name " not implemented\n"); \
    }

typedef void (*interflop_set_handler_t)(const char *name, void *function_ptr);

void interflop_set_handler(const char *name, void *function_ptr);

typedef void (*interflop_malloc_t)(ISize_t);
typedef File (*interflop_fopen_t)(const char *pathname, const char *mode,
                                  int *error);

typedef void (*interflop_panic_t)(const char *);
typedef int (*interflop_strcmp_t)(const char *s1, const char *s2);
typedef int (*interflop_strcasecmp_t)(const char *s1, const char *s2);
/* Do not follow libc API, use error pointer to pass errno result instead */
/* error = 0 if success */
typedef long (*interflop_strtol_t)(const char *nptr, char **endptr, int *error);
typedef double (*interflop_strtod_t)(const char *nptr, char **endptr,
                                     int *error);
```

# Interfaçage de la glibc : interflop-stdlib

Interface commune des fonctions de la glibc utilisées dans Verificarlo et Verrou

1. Type abstrait pour chacune des fonctions (uniformisation signatures)
2. **Une variable globale (pointeur de fonction) initialisée à NULL**
3. Un handler pour initialiser les fonctions depuis les **frontends**

## 2. Une variable globale (pointeur de fonction) initialisée à NULL

```
#define SET_HANDLER(F)
    else if (__string_equal(name, #F)) interflop_##F =
        (interflop_##F##_t)function_ptr;

interflop_malloc_t interflop_malloc = Null;
interflop_fopen_t interflop_fopen = Null;
interflop_panic_t interflop_panic = Null;
interflop_strcmp_t interflop_strcmp = Null;
interflop_strcasecmp_t interflop_strcasecmp = Null;
interflop_strtol_t interflop_strtol = Null;
interflop_strtod_t interflop_strtod = Null;
```



# Interfaçage de la glibc : interflop-stdlib

Interface commune des fonctions de la glibc utilisées dans Verificarlo et Verrou

1. Type abstrait pour chacune des fonctions (uniformisation signatures)
2. Une variable globale (pointeur de fonction) initialisée à NULL
3. **Un handler pour initialiser les fonctions depuis les frontends**

### 3. Un handler pour initialiser les fonctions depuis les outils

```
void vfc_set_handlers(const char *token, void *handle) {  
    interflop_set_handler_t set_handler = (interflop_set_handler_t)load_function(  
        token, handle, "interflop_set_handler");  
    set_handler("getenv", getenv);  
    set_handler("sprintf", sprintf);  
    set_handler("strerror", strerror);  
    set_handler("gettid", get_tid);  
    set_handler("fopen", fopen);  
    set_handler("strcasecmp", strcasecmp);  
    set_handler("vwarnx", vwarnx);  
    set_handler("fprintf", fprintf);  
    set_handler("exit", exit);  
    set_handler("vfprintf", vfprintf);  
    set_handler("malloc", malloc);  
}
```

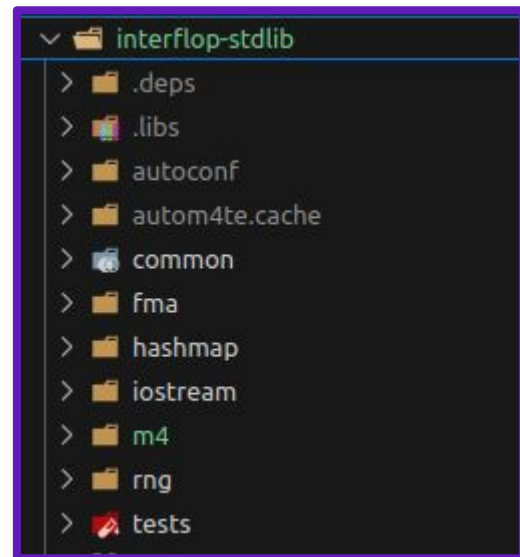
```
Int _interflop_valgrind_strcmp(const String s1, const String s2) {  
    return VG_(strcmp)(s1, s2);  
}  
  
Int _interflop_valgrind_strcasecmp(const String s1, const String s2) {  
    return VG_(strcasecmp)(s1, s2);  
}  
  
/* Do not follow libc API, use error pointer to pass errno result instead */  
/* error = 0 if success */  
Long _interflop_valgrind_strtol(const String nptr, String *endptr, Int *error) {  
    // VG_(strtoll10) returns 0 if no number could be converted,  
    // and 'endptr' is set to the start of the string.  
    *error = 0;  
    long val = VG_(strtoll10)(nptr, endptr);  
    if (val == 0 && *endptr == nptr) {  
        *error = 1;  
    }  
    return val;  
}
```

# Factorisation du code commun

## 1. Bibliothèques/Interfaces communes entre Verificarlo et Verrou

- a. Manipulation des valeurs flottantes (`float_const.h`, `float_struct.h`)
- b. Support des FMA (`fma128`) ←
- c. Logger (utilisation du logger de Verificarlo dans Verrou)
- d. RNG (Xoshiro) ←

## 2. Migration dans `interflop-stdlib`



# FMA

## Objectif :

- Ajout du support FMA dans Verificarlo
- Déjà supporté dans Verrou

## Problème :

- Utilisation de fma128bits
  - Inclusion du fichier source de libquadmath dans Verrou
  - Licence LGPL-2.1 ( incompatible avec licence Apache-2.0 (?) utilisée par Verificarlo )

## Solution :

- Interface commune
- Utilisation builtin + libquadmath (source Verrou / bibliothèque Verificarlo)

```
float interflop_fma_binary32(float a, float b, float c);  
double interflop_fma_binary64(double a, double b, double c);  
_Float128 interflop_fma_binary128(_Float128 a, _Float128 b, _Float128 c);
```

# RNG

## Constat :

- Verificarlo et Verrou utilise tous les deux Xoshiro

## Problèmes :

- RNG states différents
- Variantes Xoshiro différentes (64, 128, 128\*, 256, ...)
- Variante vectorisée

## Solution :

- Interface commune ? (pas implémenté)

```
typedef struct Vr_Rand_ Vr_Rand;  
struct Vr_Rand_ {  
    uint64_t current_;  
    tinynt64_t gen_;  
#ifdef USE_XOSHIRO  
    xoshiro128_state_t rng128_;  
    xoshiro256_state_t rng256_;  
#endif  
    uint64_t seed_;  
    double p;  
    uint32_t count_;  
};
```

```
/* Data type used to hold information required by the RNG */  
typedef struct rng_state {  
    bool choose_seed;  
    uint64_t seed;  
    bool random_state_valid;  
    __INTERNAL_RNG_STATE random_state;  
} rng_state_t;
```

# Implémentation multi-thread

## Constat :

- Verrou (Valgrind) ne supporte pas la déclaration de variable thread local
- Verificarlo le supporte

## Problème :

- Comment supporter les deux cas dans un même backend ?

## Solution :

- Générer deux versions pour chaque backend multithreaded

## Générer deux versions pour chaque backend multithreaded

```
# Backend version with TLS enabled
libinterflop_mca_la_SOURCES = interflop_mca.c
libinterflop_mca_la_CFLAGS = \
    -I@INTERFLOP_INCLUDEDIR@/ \
    -fno-stack-protector \
    -DRNG_THREAD_SAFE \
    $(LTO_FLAGS) -Og \
    $(WARNING_FLAGS)
libinterflop_mca_la_LDLFLAGS = $(LTO_FLAGS) -Og

libinterflop_mca_la_LIBADD = \
    @INTERFLOP_LIBDIR@/libinterflop_rng.la \
    @INTERFLOP_LIBDIR@/libinterflop_fma.la \
    @INTERFLOP_LIBDIR@/libinterflop_logger.la \
    @INTERFLOP_LIBDIR@/libinterflop_stdlib.la
```

```
# Backend version with TLS disabled
libinterflop_mca_no_tls_la_SOURCES = interflop_mca.c
libinterflop_mca_no_tls_la_CFLAGS = \
    -I@INTERFLOP_INCLUDEDIR@/ \
    -fno-stack-protector \
    $(LTO_FLAGS) -Og \
    $(WARNING_FLAGS)
libinterflop_mca_no_tls_la_LDLFLAGS = $(LTO_FLAGS) -Og

libinterflop_mca_no_tls_la_LIBADD = \
    @INTERFLOP_LIBDIR@/libinterflop_rng.la \
    @INTERFLOP_LIBDIR@/libinterflop_fma.la \
    @INTERFLOP_LIBDIR@/libinterflop_logger.la \
    @INTERFLOP_LIBDIR@/libinterflop_stdlib.la
```

```
VFC_BACKENDS="libinterflop_mca.so" ./test_float 1
VFC_BACKENDS="libinterflop_mca_no-tls.so" ./test_float 1
```

**V2.1.0**

**PRISM backend : arrondie stochastique  
entièrement vectorisé**





# Gestion de la vectorisation dans Verificarlo

## Problème :

- Les backends d'arrondie stochastique n'ont pas de versions vectorielles
- La perturbation est appliquée séquentiellement à chaque élément du vecteur

➡ Important **facteurs de ralentissement** pour les codes intensif en calculs

## Objectifs :

- **Performance** (bonne vectorisation)
- **Portabilité** (support des ISA x86 et aarch64)
- **Modularité** (minimiser la duplication de code)

# PRISM : Probabilistic Rounding with Instruction Set Management

- Implémentation à **faible surcoût** de l'arrondie stochastique (SR) [\[Fasi21\]](#)
- Implémentation **vectorielle** (basée sur Highway Google)
  - Sélection de la “**meilleure**” implémentation à l'exécution
    - x86-64 : SSE4.2, AVX, AVX2, AVX-512, ...
- Implémentation d'un nouveau mode d'arrondi : Up & Down (UD)
  - Rapide mais biaisée
- Utilise l'instrumentation directe pour de meilleurs performances
  - Pas de changement de backend possible à l'exécution (~~VFC\_BACKENDS~~)

$$\circ_{\text{SR}}(x) = \begin{cases} \lfloor x \rfloor & \text{with probability } p_{\text{SR}} \\ \lceil x \rceil & \text{with probability } 1 - p_{\text{SR}} \end{cases}$$

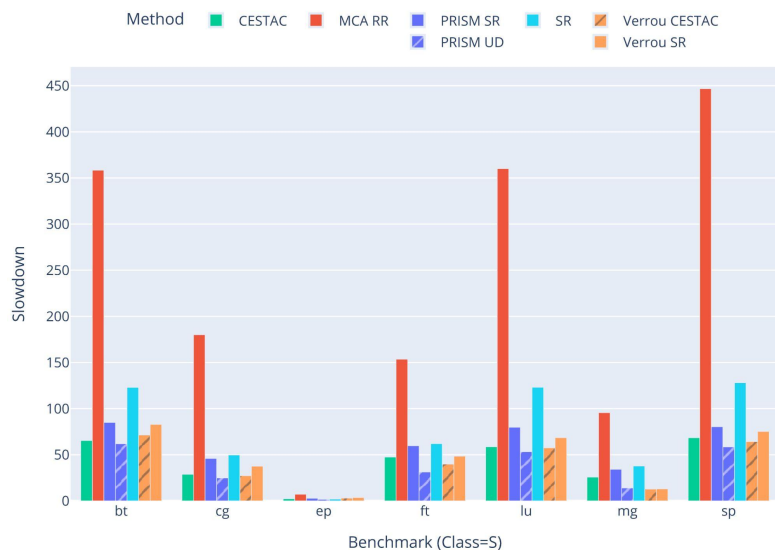
where  $p_{\text{SR}}(x)$  is defined as:

$$p_{\text{SR}}(x) = \frac{x - \lfloor x \rfloor}{\lceil x \rceil - \lfloor x \rfloor}$$

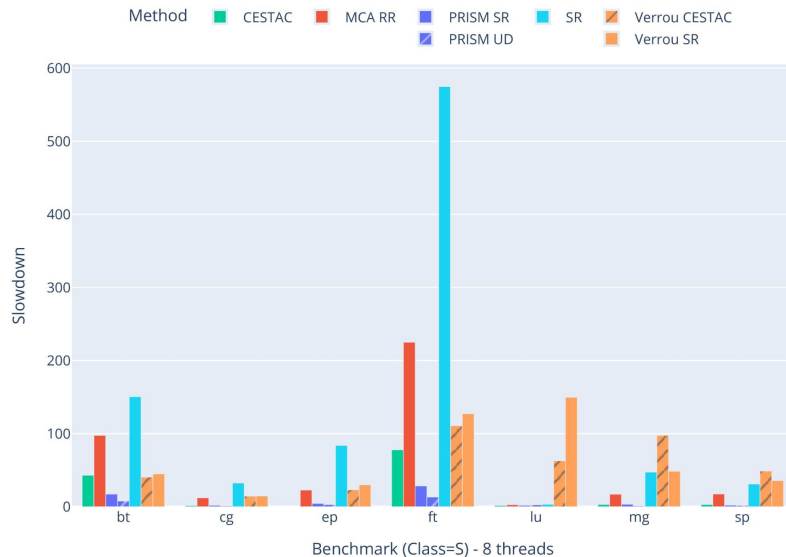
$$\circ_{\text{UD}}(x) = \begin{cases} \circ_{\text{RN}}(x) - \epsilon(x) & \text{with probability } \frac{1}{2} \\ \circ_{\text{RN}}(x) + \epsilon(x) & \text{with probability } \frac{1}{2} \end{cases}$$

where  $\epsilon(x)$  is the *unit in the last place* (ulp [12]) if  $x \neq 0$  and 0 otherwise and  $\circ_{\text{RN}}(x)$  is the IEEE-754 round-to-nearest with ties-to-even rounding mode.

# Comparaison de performances : Nas Parallel Benchmark C++ [\[Loff21\]](#)



Exécution séquentielle (référence = IEEE)



Exécution parallèle (référence = IEEE 8 threads)

**Class:** S

**BT:** Block Triangular Solver

**CG:** Conjugate Gradient

**EP:** Embarassingly Parallel

**FT:** Fast Fourier Transform

**LU:** Lower-Upper Symmetric Gauss-Seidel

**MG:** Multi-Grid Solver

**SP:** Scalar Pentadiagonal Solver

# Comparaison des performances : PyTorch

## MNIST [LeCun98]

Temps d'exécution pour 10000 images à l'inférence (single-thread)

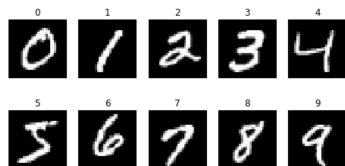


TABLE I: Comparison of runtime and overhead across instrumentation tools for MNIST in seconds

|                                        | Run 1   | Run 2   | Run 3   | Run 4   | Run 5   |
|----------------------------------------|---------|---------|---------|---------|---------|
| IEEE                                   | 14.00   | 14.02   | 14.01   | 14.01   | 14.00   |
| Verrou CESTAC                          | 4722.67 | 4278.72 | 4276.50 | 4258.39 | 4267.81 |
| Verrou SR                              | 4801.14 | 4817.18 | 4813.45 | 4960.54 | 4995.45 |
| Fuzzy PyTorch UD                       | 84.97   | 86.37   | 85.47   | 84.03   | 89.58   |
| Fuzzy PyTorch SR                       | 90.91   | 89.945  | 88.59   | 89.28   | 90.34   |
| Fuzzy UD Speedup Factor against Verrou | 55.58   | 49.53   | 50.04   | 50.67   | 47.64   |
| Fuzzy SR Speedup Factor against Verrou | 52.81   | 53.55   | 54.33   | 55.56   | 55.29   |

## FastSurfer [Henschel20]

CNN type U-net

Segmentation complète du cerveau

Temps d'exécution pour une tranche



TABLE II: Comparison of runtime and overhead across instrumentation tools for FastSurfer in seconds

|                                        | Run 1   | Run 2   | Run 3   | Run 4   | Run 5   |
|----------------------------------------|---------|---------|---------|---------|---------|
| IEEE                                   | 137.00  | 132.50  | 137.00  | 137.50  | 141.25  |
| Verrou CESTAC                          | 9953.5  | 9954.2  | 9813.4  | 10074.4 | 10004.8 |
| Verrou SR                              | 9247.2  | 9183.4  | 9216.2  | 9216.8  | 9199.2  |
| Fuzzy PyTorch UD                       | 1080.70 | 1079.84 | 1076.08 | 1075.77 | 1073.61 |
| Fuzzy PyTorch SR                       | 955.52  | 956.17  | 955.41  | 954.03  | 955.93  |
| Fuzzy UD Speedup Factor against Verrou | 9.21    | 9.27    | 9.36    | 9.12    | 9.27    |
| Fuzzy SR Speedup Factor against Verrou | 9.68    | 9.60    | 9.65    | 9.66    | 9.62    |

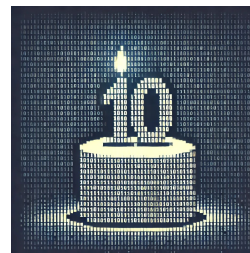
# Conclusion et perspective

- Proposition **interface commune** pour Verificarlo et Verrou
  - Permet d'utiliser les backends de manière **interchangeable**
  - Intégré dans Verificarlo v2.0.0
- **Version fonctionnelle** disponible sur [github.com/interflop/](https://github.com/interflop/)
  - Différente de la version actuelle dans Verificarlo
  - Utilise une version non-MAJ de Verrou (commit 40506202 - Oct 2022)
- **Version vectorisée** pour l'arrondie stochastique
  - **Faible surcoût** comparé a non-instrumentation (~**x10**)
  - Support multi architecture/ISA

## Travaux futures

- Compromis entre modularité/réutilisabilité et performance
  - Léger surcoût pour Verrou dans l'état actuel
- Étendre le support des **formats** et **instructions**
  - petit formats (bfloat16, fp16, fp8)
  - intrinsics (llvm.vector.reduce.\* / llvm.matrix.multiply.\*)
- Les codes GPU
  - Outils pour CPU   Outils pour GPU !?

# ✓erificarlo



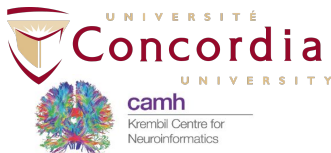
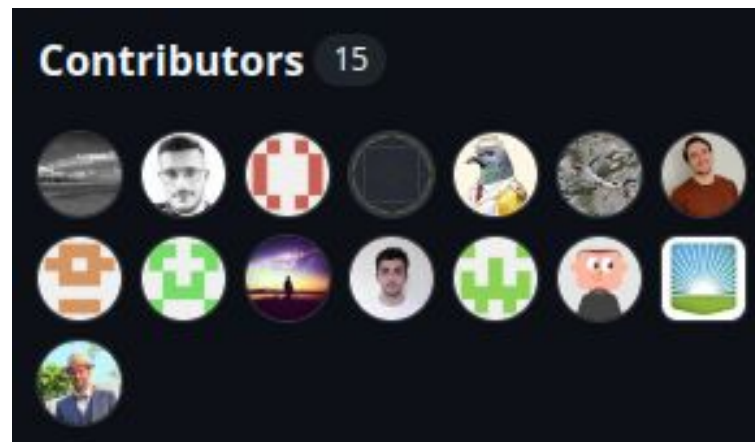
Pablo de  
Oliveira Castro



Eric Petit



Yohan  
Chatelain



Supplément



# Bibliothèque Highway de Google

- Bibliothèque C++ avec DSL pour exprimer les **instructions vectorielles**
- Mode statique avec génération de code pour les architecture ciblées
- Mode dynamique avec sélection automatique de la meilleur architecture à l'exécution
- Large gamme d'architecture et ISA supportées :
  - Armv7+ : Neon et SSE
  - IBM Z : z14, z15
  - POWER : PPC8, PPC9, PPC10
  - RISC-V : RVV
  - WebAssembly : WASM
  - x86 : SSE2, SSSE3, SSE4, AVX2, AVX3 (AVX-512F), AVX3\_DL, AVX3\_ZEN4, AVX3\_SPR

➡ Très bon compromis entre  
**performance, portabilité** et **modularité** !

```
template <class D, class V = hn::VFromD<D>, typename T = hn::TFromD<D>>
HWY_FLATTEN void twosum(const D d, const V a, const V b, V &sigma, V &tau) {
    dbg::debug_msg("\n[twosum] START");
    dbg::debug_vec(d, "[twosum] a", a);
    dbg::debug_vec(d, "[twosum] b", b);

    sigma = hn::Add(a, b);
    const auto a_p = hn::Sub(sigma, b);
    const auto b_p = hn::Sub(sigma, a_p);
    const auto d_a = hn::Sub(a, a_p);
    const auto d_b = hn::Sub(b, b_p);
    tau = hn::Add(d_a, d_b);
    tau = hn::IfThenElseZero(hn::IsFinite(sigma), tau);

    dbg::debug_vec(d, "[twosum] sigma", sigma);
    dbg::debug_vec(d, "[twosum] tau", tau);
    dbg::debug_msg("[twosum] END\n");
}
```



# Comparaison des performances sur PyTorch

## MNIST [LeCun, 1998] et FastSurfer [Henschel, 2020]

### MNIST :

Classification de 10000 chiffres manuscrits

Test inférence

### FastSurfer :

Segmentation complète du cerveau, reconstruction de la surface corticale, mappage sphérique et analyse de l'épaisseur corticale à partir d'IRM

#### Architecture CNN :

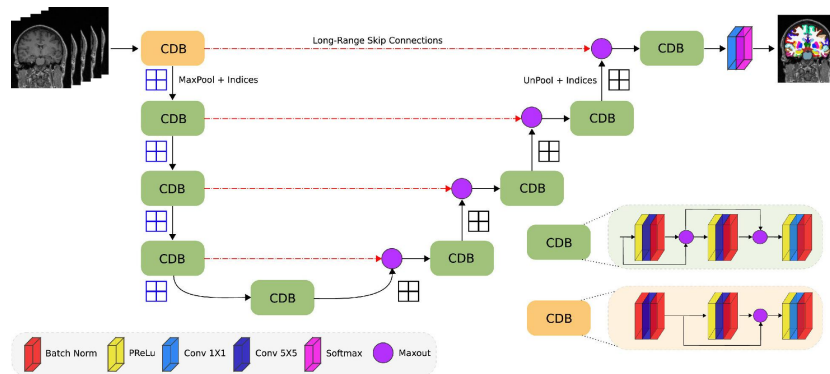
- Basée sur QuickNAT [Roy, 2019] avec une structure d'encodage/décodage de type U-net
- Composée de 3 réseaux 2D entièrement convolutionnels pour différentes orientations de coupes.

#### Performance :

- Dépasse les méthodes à la fine pointe pour la qualité de la segmentation
- Généralisation aux ensembles de données inconnus avec une haute fiabilité test-retest

#### Test :

5 sujets, T1w 256x256x256 résolution 1 mm, 3 tranches seulement (axiale, coronale, sagittale)



Architecture de FastSurfer [Henschel, 2020]